Money, Minutes and Model Checking: an AI Assistant Checks ATM Timing in Uppaal

Author Names

Institution

Abstract. A bank engineer asked an AI assistant, connected to Uppaal through a tool interface, to build a timed-automata model of an ATM network and verify its time constraints. Within one working session the assistant built and edited the model, wrote and ran 22 queries, and explained the results. Verification uncovered a lost-money scenario before it was fixed, exposed a conflict between that fix and a session deadline, and showed at which network size a timeout stops being safe. We summarise the model and results and reflect on what the session suggests for integrating formal methods into engineering practice with AI assistance.

1 Setting

The engineer stated one request: "create a timed automata for ATM" to verify time constraints of a planned ATM network with timed automata [1]. The assistant had direct access to the open Uppaal [2] editor: it could read the model, replace it, add or change locations, edges, declarations and queries, check syntax, and run verification with time and memory limits. The engineer saw every change on the canvas, asked questions, and made the decisions about requirements and scope. This paper reports the session as a demonstrator of that division of labour, with an eye on two iFS themes: integrating methods into engineering practice, and AI for software engineering.

2 The Model

The network has N ATMs and one bank authorisation server. Three templates are instantiated: `ATM(id)` (Fig. 1), a `Bank` that serves requests in arrival order and needs 1–4 s for each, and a `Customer(id)` with a single location and a self-loop for every action. The customer can therefore do anything at any time, including nothing, so every verified property holds for all customer behaviours. Each ATM has a step clock `t` and an observer clock `s` for the whole session; the bank has one clock `b`.

The timing constraints are bounds, not fixed delays:

```
const int PIN_TIMEOUT = 30, AMOUNT_TIMEOUT = 30; // customer
const int NET_TIMEOUT = 10;                      // ATM waits for bank
const int BANK_MIN = 1, BANK_MAX = 4;            // bank, per request
const int CARD_TIMEOUT = 20, CASH_TIMEOUT = 20; // retain / retract
const int DISPENSE_MIN = 2, DISPENSE_MAX = 5;   // dispenser
const int EJECT_MIN = 1, MAX_SESSION = 200;     // eject, session bound
```

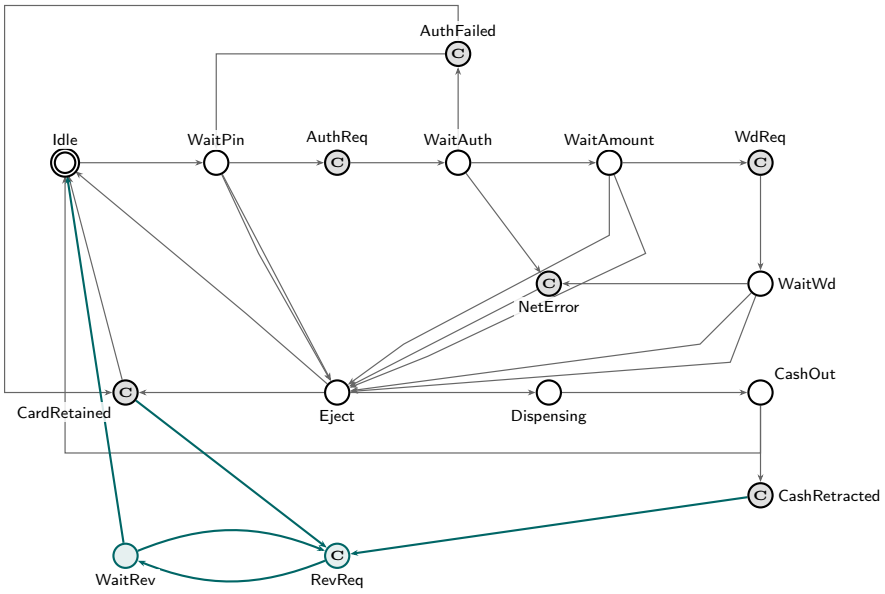


Fig. 1. The `ATM` template at the end of the session, generated from the model file using UPPAAL's own layout (edge labels omitted). Grey locations marked C are committed; the double circle is initial. Highlighted: the debit-reversal routine added during the session.

When the engineer found the ATM automaton "a bit complex", the assistant sorted its locations by purpose. Seven are *deadline locations*: an invariant such as `t ⩽ PIN_TIMEOUT` with a timeout edge guarded by `t ⩾ PIN_TIMEOUT` means "fires exactly at the deadline". Two committed locations exist only because an edge carries one synchronisation (receive from the customer, then send to the bank). The remaining locations and variables are observers that let queries name outcomes. One design choice came from the domain rather than the formalism: the card is returned before the cash, as in real ATMs. `EJECT_MIN` also has a formal job: it rules out zero-time cycles, without which the liveness property `!ATM(0).Idle --> ATM(0).Idle` fails on Zeno runs.

3 Verification-Driven Changes

A failing property before the fix. The engineer asked for the debit to be reversed when a card is retained. Before changing the behaviour, the assistant added a ledger `debited[N]` and the property

```
A[] forall(i:id_t) ATM(i).Idle imply !debited[i]   // no debit left behind
```

which failed: an approved withdrawal followed by an untaken card ended the session with the account debited and no cash paid out. Only then was the reversal routine added (`RevReq`, `WaitRev`, a `REVERSAL` request type in the bank), after

Table 1. Selected results for the final model ($N = 2$). 21 of 22 queries hold.

Property	Result
No deadlock; every started session ends; cash only after a correct PIN	hold
No debit left without cash paid out or reversed	holds
Bank answers every request within N·`BANK_MAX`; no network timeout	hold
Fastest / slowest time from card insertion to cash presented	5 s / 177 s
Worst-case session (`sup`)	205 s
Every session ends within `MAX_SESSION` = 200 s	**fails**

Table 2. Effect of the number of ATMs. "≈": from the path formula $165 + 5W$, which matches the verified values for $N = 1, 2$.

N	bank wait W	timeouts at 10 s	money safety	worst session	verification per query
1	4 s	no	holds	185 s	<0.01 s
2	8 s	no	holds	205 s	0.05–0.3 s
3	12 s	yes	**fails**	≈225 s	25–42 s, ≈600 MB
4	16 s	yes	–	≈245 s	>50 s, unfinished

which the property holds. Writing the property first turned an informal request into a regression check.

A fix that breaks a deadline. Reversing retracted cash as well kept money safety but added one bank round trip to the worst path. The worst session, computed with `sup{!ATM(0).Idle}: ATM(0).s`, grew from 197 s to 205 s, so the requirement that every session ends within `MAX_SESSION` now fails. The assistant listed three resolutions (relax the bound, shorten `CASH_TIMEOUT`, or send reversals asynchronously); the engineer deferred the decision. A functional fix had a timing cost that nobody had asked about, and the model made it visible at once.

4 Timing and Scale

The engineer asked whether the constants were "worst case scenarios". They are not: they bound a dense set of behaviours that UPPAAL explores symbolically, `A[]` properties hold for all of them, and `sup`/`inf` queries return the worst and best case. This question, rather than any query, was where the engineer's mental model and the formal semantics differed.

Table 2 shows the scaling study. With a FIFO bank the worst wait is $4N$ s, so a 10 s timeout supports at most two ATMs. At $N = 3$ verification found more than slow answers: a withdrawal that times out after the bank has debited the account loses money, and requests abandoned by timed-out ATMs overflow the bank queue. Real ATMs guard against this with timeout reversals and request identifiers; the model has neither and is safe only while timeouts are impossible. Verification cost grew from 0.05 s to about 35 s per query between $N = 2$ and $N = 3$, and $N = 4$ did not finish within 50 s.

5 Reflections

The engineer decided, the assistant formalised. The engineer never wrote UPPAAL syntax but made every decision that changes what the model means: which reversals to add, which issues to defer (the session bound, a daily withdrawal limit), and what counts as realistic. The engineer explicitly valued that the model followed real ATM practice.

Properties first. With verification one call away, the assistant could show a property failing, change the model, and show it passing, giving the engineer evidence rather than assurance.

Explanation is part of the method. Two of the engineer's questions were about the model rather than the system: why the automaton is complex, and what the constants mean. Answering them in terms of deadlines, committed locations and symbolic exploration is what made the results usable for the engineer.

The tool interface shapes the work. The first model was written whole; later changes used targeted edits so the engineer's layout and undo history survived. Edits address elements by position, so deletions had to run from the highest index down, and edges added programmatically had no label positions, which hurt the diagram. Verification calls are synchronous and UPPAAL accepts one request at a time: a 1000 s constant, chosen by the assistant to switch timeouts off, blew up the zone graph and blocked the tool for about 25 minutes. Small constants (a timeout of $4N + 1$ s) and single-query runs with time limits fixed this.

Results are conditional. The timing values were plausible defaults chosen by the assistant, not measurements from the bank, and results beyond $N = 3$ are extrapolated; the assistant marked verified and estimated numbers separately.

Outlook. The open issues point to standard techniques: timeout reversals with request identifiers, a compositional argument with one detailed ATM and a worst-case load generator, and SMC for average-case questions at realistic network sizes. This paper, its figure and an HTML report were generated by the assistant from the same model file.

References

1. Alur, R., Dill, D.L.: A theory of timed automata. Theor. Comput. Sci. **126**(2), 183–235 (1994)
2. Behrmann, G., David, A., Larsen, K.G.: A tutorial on UPPAAL. In: SFM-RT 2004. LNCS, vol. 3185, pp. 200–236. Springer (2004)